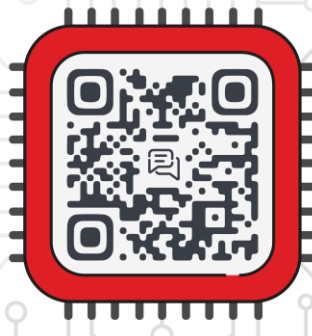




نظري

4

10

1200 sp

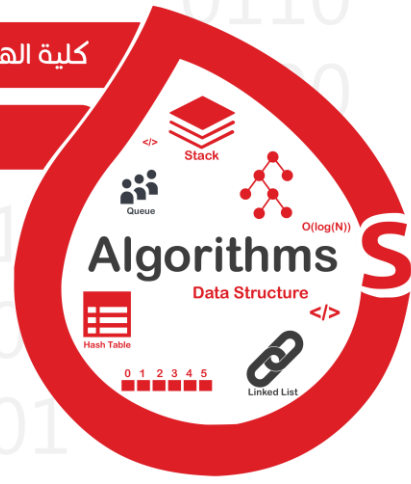
كلية الهندسة المعلوماتية

السنة الثانية

البحث الثنائي:

المفهوم و التطبيقات

د. أبي صندوق



RB Informatics; 08/11/2025

خوارزميات 1

في المحاضرة السابقة تحدثنا عن طريقة التفكير Decrease and Conquer و أشكالها، و اعتمدنا في حلنا للمسائل على مبدأ Decrease by one. وفي هذه المحاضرة سنتطرق للحديث عن خوارزمية جديدة و هي طريقة تمكننا من إيجاد حل لبعض المسائل حيث تعتمد على تقسيم المجال الذي نعمل عليه في المسألة الى قسمين، و هذه الطريقة تسمى بـ:

البحث الثنائي Binary Search

هي خوارزمية بحث تستخدم لإيجاد موقع عنصر معين من بين مجموعة من العناصر المرتبة تصاعدياً أو تنازلياً، حيث يتم في كل خطوة تقسيم نطاق البحث إلى نصفين، ثم تحديد أي نصف يحتوي على العنصر المطلوب، وإهمال النصف الآخر. تستمر العملية حتى يُعثر على العنصر أو يُصبح نطاق البحث فارغاً.

ملاحظة هامة جداً:

لا تعمل الخوارزمية بشكل صحيح إذا لم تكن مجموعة العناصر مرتبة تصاعدياً أو تنازلياً.

و الآن مع بعض الأمثلة لتوضيح الخوارزمية:

المثال 1:

تخيل أن لديك قاموساً (معجم) وتريد البحث عن كلمة معينة. بدلاً من البحث من البداية حتى النهاية (وهو ما نسميه البحث الخطي)، يمكنك استخدام طريقة أكثر ذكاءً - **البحث الثنائي**.

الفكرة الأساسية:

البحث الثنائي: افتح القاموس من المنتصف، و انظر إذا كانت الكلمة التي تبحث عنها قبل أو بعد هذه الصفحة، ثم كرر العملية على النصف المناسب.

لنأخذ مثلاً بسيطاً..

ليكن لدينا مصفوفة كلمات نريد البحث عن الكلمة التي تبدأ بحرف 'c':

["apple", "banana", "cherry", "date", "fig", "kiwi", "strawberry"]

خطوات البحث عن أول كلمة تبدأ بالحرف المطلوب:

1. نريد تقسيم مجال البحث الى النصف : نحسب الفهرس الوسط وهو $(6 + 0)/2 = 3$.
2. نذهب الى الكلمة التي توجد في الموضع 3 فنجد أنها تبدأ بالحرف d (date).
نلاحظ أن الحرف c أصغر من الحرف d وفقاً للترتيب الابجدي، هذا يعني أن الكلمات التي تبدأ بالحرف c يجب أن تكون في النصف الأيسر.
3. نضبط المجال مرة أخرى لنكمل بحثنا ونجعل المجال الجديد من 0 إلى 2.
4. الفهرس الأوسط الجديد: $(2 + 0)/2 = 1$.
5. نذهب إلى الكلمة التي توجد في الموضع 1 فنجد أنها تبدأ بالحرف b (banana).
نلاحظ أن الحرف c أكبر من الحرف b، هذا يعني أن الكلمات التي تبدأ بالحرف c يجب أن تكون في النصف الأيمن من المجال.
6. أصبح المجال الآن من 2 إلى 2 وهي الكلمة المطلوب إيجادها: Cherry.

المثال 2:

لدينا مصفوفة مكونة من 10 عناصر من الأعداد المرتبة تصاعدياً نريد أن نبحث فيها عن الرقم 37 باستخدام خوارزمية البحث الثنائي.

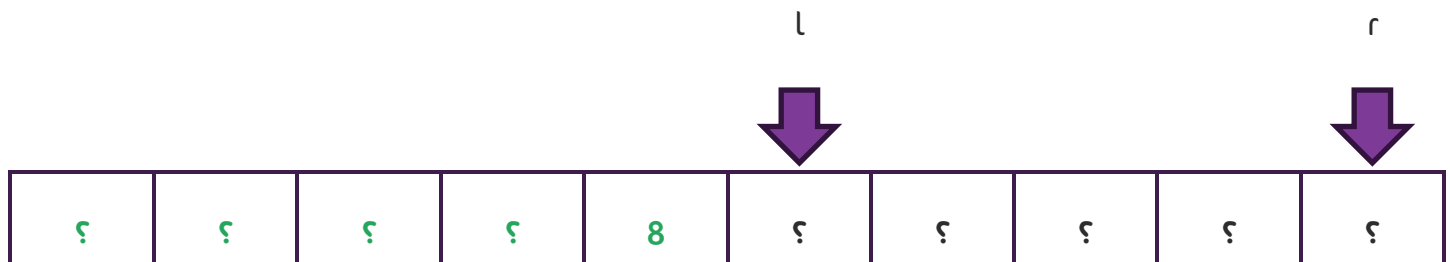
0	1	2	3	4	5	6	7	8	9
؟	؟	؟	؟	؟	؟	؟	؟	؟	؟

سنقوم بتسمية المؤشرات لسهولة التعامل مع الفهارس $Left \rightarrow l=0$ ، $Right \rightarrow r=9$.

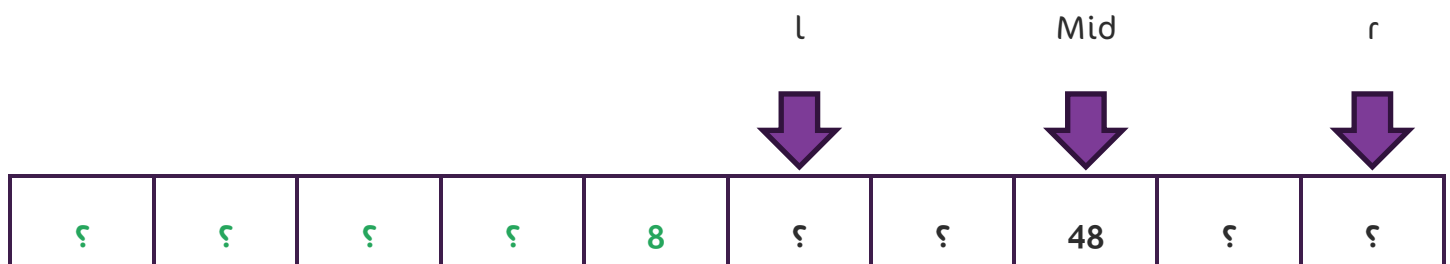
نقوم الآن بحساب منتصف المجال: $Mid=(r+l)/2=4$

l					Mid					r
↓					↓					↓
؟	؟	؟	؟	8	؟	؟	؟	؟	؟	؟

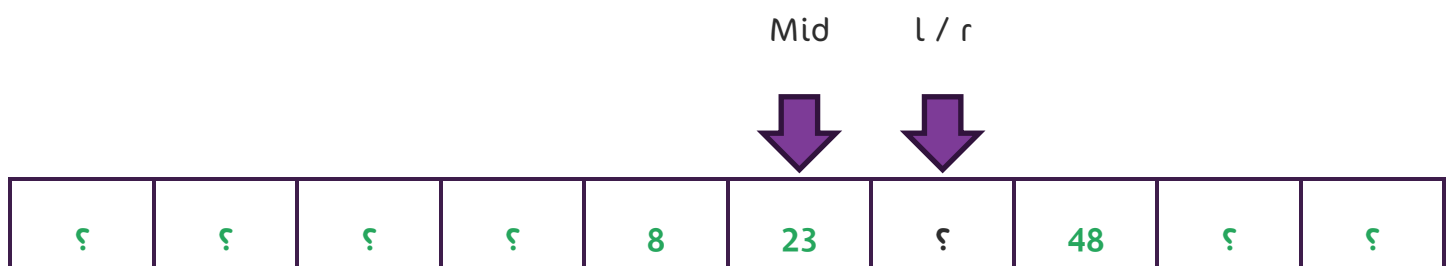
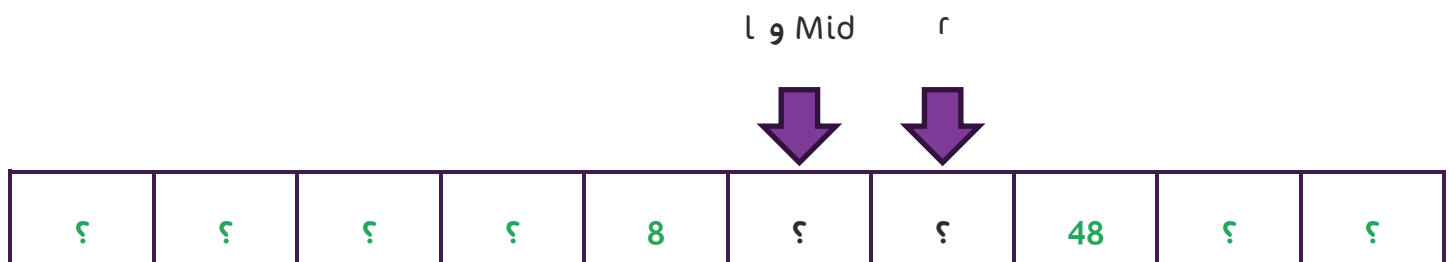
8 > 37 إذاً ننتقل للبحث في المجال الأيمن و نهمل الايسر فلا حاجة لنا به، و يصبح $l = \text{Mid} + 1$



$$\text{Mid} = (\text{l} + \text{r}) / 2$$



$$r = \text{Mid} - 1$$



باستخدام طريقة البحث هذه لقد قمنا باختصار الكثير من عمليات المقارنة بواسطة تصغير المجال في كل مرة حيث كان يسير بهذا الشكل : $N \rightarrow N/2 \rightarrow N/4 \rightarrow \dots \rightarrow 1$
حيث أن N هو عدد عناصر المصفوفة، وهنا $N=10$.

طريقة كتابة الكود:

```
public class BinarySearchExample {
    public static int binarySearch(int[] arr, int target) {
        int left= 0;
        int right = arr.length - 1;
        while (l<= r) {
            int mid = (r+l) / 2;
            if (arr[mid] == target) {
                return mid //وجدنا العدد
            }
            else if (arr[mid] < target) {
                l = mid + 1; //نبحث في النصف الأيمن
            }
            else {
                r = mid - 1; //نبحث في النصف الأيسر
            }
        }
        return -1; //العدد غير موجود
    }
    public static void main(String[] args) {
        int [] numbers = {1, 2, 5, 6, 8, 23, 37, 48, 50, 65};
        int target = 37;
        int result = binarySearch(numbers, target);
        if(result != -1) {
            System.out.println("العدد ☒ " + target
                + " موجود في الفهرس");
        }
        else {
            System.out.println("العدد ☐ " + target
                + " غير موجود في المصفوفة");
        }
    }
}
```

⚡ التعقيد الزمني:

في كل مرة نقوم بتقسيم المصفوفة إلى نصفين ⇐ التعقيد في الكود السابق هو $O(\log n)$ ، (عدد الخطوات ينمو ببطء شديد مع زيادة عدد المدخلات).

كلما نسمع كلمة تقسيم مستمر لمجال معين في الخوارزمية فوراً منفكر باللوغاريتم 😊

٤ قد يخطر سؤال لدى البعض... هل يوجد بحث ثلاثي و رباعي و...؟؟؟

في الحقيقة يوجد... و ها هي عائلة البحث:

1. تقسيم المجال إلى نصفين (Binary Search) البحث الثنائي.
2. تقسيم المجال إلى ثلاثة أجزاء متساوية (Ternary Search) البحث الثلاثي.
3. تقسيم المجال إلى أربعة أجزاء متساوية (Quaternary Search) البحث الرباعي.

سنقوم بمقارنة سريعة بين الأنواع الثلاث:

البحث الرباعي	البحث الثلاثي	البحث الثنائي
الفكرة: يقسم المجال إلى أربعة أجزاء متساوية كل مرة.	الفكرة: يقسم المجال إلى ثلاثة أجزاء متساوية كل مرة.	الفكرة: يقسم المجال إلى نصفين في كل خطوة.
نحسب ثلاث نقاط فاصلة: mid1 و mid2 و mid3.	نحسب موقعين: mid1 و mid2.	نقارن الكلمة المطلوبة بالمنتصف، و نقرر هل نبحث في النصف الأيمن أو الأيسر.
نحدد في أي ربع تقع الكلمة المطلوبة.	نحدد أي جزء يحتوي على الكلمة و نستبعد الباقي.	مثال القاموس: إذا كنت تبحث عن كلمة تبدأ بالـ "م"، تفتح القاموس من المنتصف، -إذا كانت الكلمة في المنتصف تبدأ بحرف "م" - نذهب للنصف الأيمن. -إذا بعده - نذهب للنصف الأيسر.
التعقيد: $O(\log_4 n)$	مثال القاموس: افتح القاموس على الثلث الأول، ثم على الثلث الثاني، إذا كانت الكلمة المطلوبة بينهما ← نبحث في هذا الجزء فقط.	التعقيد: $O(\log_2 n)$
هناك 3 مقارنات في كل خطوة، لذا عملياً ليس أسرع من البحث الثنائي.	التعقيد: $O(\log_3 n)$	

ملاحظة:

رغم أن الخوارزميات الثلاث كلها تقلل مجال البحث؛ إلا أن البحث الثنائي هو الأفضل عملياً لأنه يحقق توازناً مثالياً بين عدد المقارنات و السرعة و سهولة التنفيذ.

انتباه !!!

البحث الثلاثي و الرباعي نادراً ما نستخدمهما، و غير مطالبين بطريقة التكويد.

ملاحظة أريسية:

- لدينا مسألة ما، قمنا بتصغيرها ثم حلّ الجزء الصّغير بعدها تعميم الحل على الجزء الأكبر، هذه الطّريقة تدعى: Decrease and Conquer، أمّا عند تقسيم المسألة لعدّة أقسام صغيرة ثمّ حلّ كلّ جزءٍ على حدّى بعدها تجميع الحلول ، هذه الطريقة تدعى: Divide and Conquer.
- خوارزمية Binary Search تعتمد مبدأ Decrease by Factor، حيث إنّها تقلّل حجم المشكلة بمعامل ثابت في كل خطوة، فمثلاً لو كان لدينا 1000000 عنصر فإن البحث الخطي $O(n)$ يحتاج مليون خطوة، أمّا البحث باستخدام المعامل الثابت فإنّه يحتاج فقط إلى 20 خطوة تقريباً.

عند تصغير المسألة بطريقة عشوائية (و هي مسائل نادرة) نكون قد استخدمنا Decrease by variable size، و هي النوع الثالث من Decrease and Conquer.

✍ مثال على Decrease by variable size:

✍ إيجاد القاسم المشترك الأكبر (GCD).

الطريقة الأولى:

الطرح المتكرر (Subtraction method):

إذا كان لدينا عددين a و b ، نقوم بطرح العدد الأصغر من الأكبر مراراً حتى يتساويان، و يكون الناتج هو الـ GCD.

24	6	
18	6	$24 - 6 = 18$
12	6	$18 - 6 = 12$
6	6	$12 - 6 = 6$

Code:

```

1 public class GCDsubtraction {
2     public static int gcd (int a, int b) {
3         while (a != b) {
4             if (a > b)
5                 a = a - b;
6             else
7                 b = b - a;
8         }
9         return a;
10    }
11 }
```

ملاحظة:

return a هي نفسها return b.
لأنه في النهاية سنصل لـ $b=a$.

$$T(n) \Rightarrow O(\max(a, b))$$

الطريقة الثانية:

خوارزمية إقليدس (Euclidean algorithm):

العملية الأساسية هي $GCD(a, b) = GCD(b, a \% b)$ ، و نكرر العملية حتى يصبح باقي القسمة 0.

مثال:



Code:

```

1 public class GCD {
2     public static int gcd (int a, int b) {
3         while (b != 0) {
4             int temp = b;
5             b = a % b;
6             a = temp;
7         }
8         return a;
9     }
10 }

```



$$T(n) = O(\log(\min(a, b)))$$

الطريقة الثالثة:

باستخدام باقي القسمة بشكل مباشر (Recursive Euclidean):

← الطريقة العودية، نفس فكرة الطريقة الثانية لكن بطريقة استدعاء الدالة نفسها.

Code:

```

1 public class GCDRecursive {
2     public static int gcd (int a, int b)
3     {
4         if (b == 0)
5             return a;
6         return gcd(b, a % b);
7     }
8 }

```

$$T(n) = O(\log(\min(a, b)))$$

ملاحظات:

1. في مسألة GCD استطعنا إيجاد مسألة أصغر من الأصلية، بعدها قمنا بحل المسألة الأصغر ثم عمّمنا الحل على كامل المسألة.

■ إضافة الكريّة النيرة بالرياضيات... 😊

هل نستطيع إثبات أن حلّ الجزء الصّغير هو نفسه حلّ الجزء الكبير !!؟
(يعني $GCD(a, b) = GCD(b, a \% b)$)

الجواب: طبعاً 💧💧

بس إذا كنت مهتمّ بالإثبات الرياضي رو دورّ عليه ☺

2. تذكر أنّ الخطوات التي تحدّثنا عنها في مبرهنة إقليدس هي خطوات متسلسلة منتهية و تضمن وجود الحل...
فهي خوارزمية.

الطريقة الرابعة:

يمكن استخدام العوامل الأولية لكل عدد



$$12 = 2 \times 2 \times 3$$

$$8 = 2 \times 2 \times 2$$

GCD هو ضرب الأعداد المشتركة بينهما

$$\rightarrow 2 \times 2 = 4$$

■ نلاحظ أنه في هذا النوع من المسائل يتقلّص المجال بنسبة غير محدّدة في كل مرّة، و هذا ما يدعى بال (Decrease by variable size) و ليست مثل خوارزمية ال (Binary Search) التي كانت في كل مرة تقسم المسألة إلى نصفين بشكل ثابت.

تساؤل:

أيهما أفضل في كتابة الكود، الطريقة التكرارية ؟ أم الطريقة العودية ؟؟

1. إذا كان الهدف هو وضوح الكود و التقليل من التعقيد الفكري:

الاستدعاء الذاتي (العودية) ممتاز، خاصة للمشاكل الرياضيّة أو الشجرية، مثل:

Factorial – Fibonacci – GCD – Quick sort – Merge sort

2. إذا كان الهدف هو الأداء و الذاكرة فالعودية غير ملائمة.

مسألة: احسب القوة x^a

🔑 يوجد عدة طرق لحساب القوة...

الطريقة الأولى: Brute Force

$$x^a = \underbrace{x \times x \times \dots \times x}_a \text{ مرة}$$

Code:

```

1 public class Power {
2     public static double power (int x, int a) {
3         double result = 1;
4         int exponent = Math.abs(a);
5         for(int i = 0; i <= exponent; i++)
6             result *= x;
7         if (a < 0)
8             result = 1.0/result;
9         return result;
10    }
11 }
```



الطريقة الثانية:

$$\left(x^{\frac{a}{2}}\right)^2 = x^a$$

$$\text{نعلم أن } \left(x^{\frac{1}{2}}\right)^2 = x^1$$

$$x^{\frac{a}{2}} \cdot x^{\frac{a}{2}} \cdot x$$

$$x^{\frac{a}{2}} \cdot x^{\frac{a}{2}}$$

نميز حالتين ← a فردية
 ← a زوجية

قد نقول أن هذه مسألة بسيطة، لكن قد تكون a عدداً كبيراً جداً، أو قد تكون a عبارة عن مصفوفة.

أحد طرق كتابة الكود هي الطريقة العودية، حيث أنها تقلل من n تدريجياً حتى نصل إلى الحالة الأساسية $a^0 = 1$.

Code:

```

1 public class Power {
2     public static long power2 (long a, int n) {
3         if(n == 0)
4             return 1;
5         else if(n % 2 == 0) {
6             long result = power2(a, n/2);
7             return result*result;
8         }
9         else
10            return a*power2(a, n-1);
11     }
12 }

```

تعقيد الكود $O(\log n)$.

الحل بالطريقة التكرارية:

```

1 public class PowerCalculate {
2     public static long power (long a, int n) {
3         long result = 1;
4         long base = a;
5         while(n > 0) {
6             if(n % 2 == 1)
7                 result *= base;
8             base *= base;
9             n /= 2;
10        }
11        return result;
12    }
13 }

```

التعقيد لا يختلف عن الطريقة العودية. (أي $O(\log n)$).

ملاحظة:

العودية أو التكرارية هي طريقة لكتابة الكود و ليست طريقة تفكير.
التعقيد يتعلق بطريقة التفكير، حيث إنه لكل خوارزمية تعقيد خاص بها.

في الختام...

في الحياة العملية لا نستخدم خوارزمية إقليدس مثلاً، و لا نواجه مسائل بحث و فرز معقدة؛ و لكن ما يهمنّا هو تعلّم طريقة التفكير و ليس المسألة نفسها.